ENPH 459 - 2405 - Horns Algorithm Implementation

Chayce Ross and Sarah Gauthier

6th April 2024

Introduction and Theory

In order to map the coordinate system for the fiducials in the CT scans x_l ("CT fiducials") to the fiducials in our game world x_r ("physical fiducials") we used Horns algorithm (Robotics Knowledgebase, 11th May 2020). The goal of Horn's algorithm is to minimize the error (e_i) in distance between corresponding fiducials in two coordinate systems.

Steps

1. Find the centroid of each coordinate system where each fiducial has equal weighting.

$$\overline{x} = \frac{1}{N} \sum_i x_i$$

2. Create new coordinate systems x' that are shifted by the centroids.

$$x'_i = x_i - \overline{x}_i$$

3. Find the scale, by the ratio of the norm squared of each shifted coordinate system.

$$s = \sqrt{\left(\sum_i |x'_{l,i}|^2\right) \Big/ \left(\sum_i |x'_{r,i}|^2\right)}$$

4. Finally we find the rotation matrix, by minimizing the resulting error function - $F(\theta, \phi, \psi)$ (derivation can be found in Appendix 1).

$$R = \operatorname{argmin} 2 \left(\sqrt{\left(\sum_i |x'_{l,i}|^2\right) \left(\sum_i |x'_{r,i}|^2\right)} - \sum_i x'_{l,i} \cdot \left(R x'_{r,i}\right) \right)$$

Optimization of Rotation Function

The .NET standard library does not provide a package for scientific analysis. While there are .NET packages that do provide ways of optimizing functions, they have proven hard to include in our Unity projects because many do not target Android. Accordingly, we implemented our own optimizer $Q\{E(\theta, \phi, \psi)\} \rightarrow (\theta, \phi, \psi)$. Our algorithm uses a **prune and search** method to iteratively select candidate spaces. This drastically reduces the time complexity in searching for the minimizing rotation matrix. If we were to test every possible solution with an precision of Δt degrees of (θ, ϕ, ψ) the search space S would be $S_\theta \cdot S_\phi \cdot S_\psi = \left(\frac{S_0}{\Delta t}\right)^3$ where S_0 is the breadth of the search space.

Prune and Search

Our prune and search algorithm breaks the search space (S_i) into an $N \times N \times N$ tensor $({}_i\mathbf{T})$, where each element is identified by its centre: ${}_i\mathbf{T}_{(x,y,z)} = {}_i(\Theta_x, \Phi_y, \Psi_z)$ with $(x, y, z) \in [0, 1, \ldots, N)$. We then create a new tensor $({}_i\mathbf{E})$, representing the corresponding errors by passing this tensor through our error function: ${}_i\mathbf{E} = F({}_i\mathbf{T})$. The algorithm then selects the P smallest elements $e_0, e_1, \ldots, e_{P-1}$ from ${}_i\mathbf{E}$, and returns their corresponding arguments ("candidates"): $G = \{G_j | j \in [0, 1, \ldots, P)\}$. These define P new search spaces each with size S_i/N. Successive iterations of the algorithm run similarly on each element G_j of ${}_iG$; however, the smallest elements $e_0, e_1, \ldots, e_{P-1}$ are selected from all $\mathbf{E}_j$. i.e. Each iteration only produces P candidates for the next iteration. This is repeated for M iterations, i.e. $i \in [0, 1, \ldots M)$, where the algorithm's precision is $\Delta t = \frac{S_0}{N^M}$. On the last iteration the the candidate that returns the smallest error is returned as the solution to the optimization.

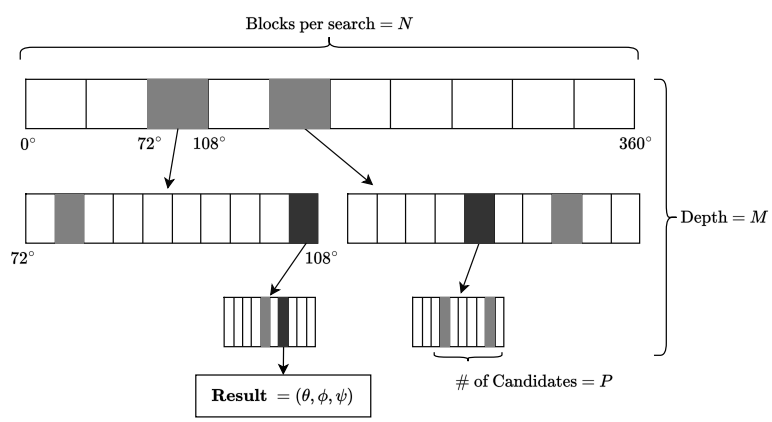

Figure 1: Tree diagram of rotation optimizer (analogy in 1 dimension). Note that the grey blocks are the minima values of each sequence.

The time complexity of this algorithm is:

$$\mathcal{O}(N^3 \cdot P \cdot M) = \mathcal{O}\left(PN^3 \cdot \frac{\log(S_0/\Delta t)}{\log N}\right)$$

When we consider for $S_0 = 360°$, $\Delta t = 0.1$, $N \sim \mathcal{O}(10)$, $P \sim 10$ the time complexity is about $\mathcal{O}(10^5)$ rather than the naive solution which was about $\mathcal{O}(10^9)$.

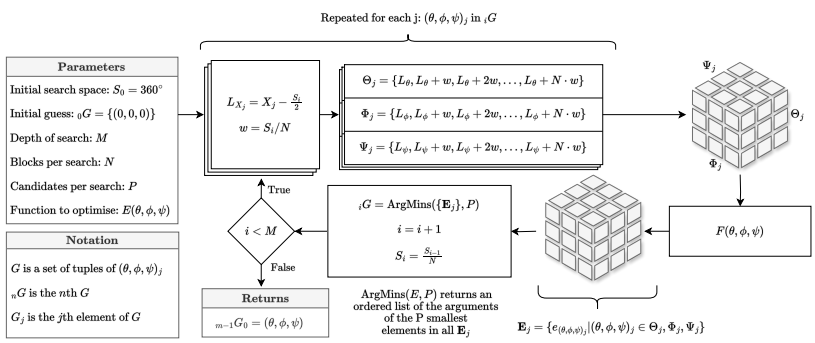

Figure 2: Flowchart of rotation optimizer.

References

Robotics Knowledgebase. (11th May 2020). *Registration techniques in robotics.* https://roboticsknowledgebase.com/wiki/math/registration-techniques/